SIMATS ENGINEERING

Assessment Tool 3: Reverse Engineering Task

Course Name: Cloud Computing and Big Data Analytics
Course Code: CSA15

Course Outcome Covered

CO4: Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)

Dave's Taxonomy: Articulation (Level 4) | Question Count: 5 | Total Marks: 30

Code of Conduct

I **DEENADAYALAN P (Reg. No. 192521135)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P DEENADAYALAN

Approach: for each scenario below, the described system is treated as a black box — only the behaviour and requirements stated in the prompt are known. Each component, tool choice, and design decision is inferred from that evidence and explicitly justified, rather than assumed.

Q1. Web, Mobile & Transaction Analytics Platform (6 Marks)

Reverse-Engineered Understanding

Billions of clicks, mobile interactions, and transactions every day rules out a single relational database from the start — no single-node system sustains that write volume. The scenario's mix of structured transactions and semi-structured interaction logs, plus a requirement for both real-time reporting and deep analysis, points to a classic lambda-style big data platform: a distributed ingestion layer feeding both a batch path and a speed (streaming) path into a shared distributed store.

Inferred Components and Technical Evidence

Inferred Component	Likely Technology	Evidence / Reasoning from Scenario
Ingestion framework	Apache Kafka (or Flume for log-style data)	"Billions ... every day" implies a durable, high-throughput log-based ingestion layer that can buffer producers from slower downstream consumers
Distributed storage	HDFS or cloud object storage (S3/ADLS) as a data lake	Mixed structured/semi-structured data at this scale is normally landed raw first, then organised — a hallmark of a data-lake pattern rather than a fixed schema database
Structured vs. semi-structured separation	Separate zones/tables — relational-style tables for transactions, JSON/Parquet partitions for clickstreams	Transactions are naturally tabular (schema-on-write), while click/interaction logs are commonly JSON — keeping them in different formats/zones avoids forcing one schema onto both
Batch processing	Apache Hadoop MapReduce / Spark batch	Deep historical analysis (trends, reporting) over billions of records is a classic large-scale batch workload
Stream processing	Spark Structured Streaming or Apache Flink	"Real-time analytics and reporting" cannot wait for a batch cycle, so a

		parallel low-latency engine is almost certainly present
Indexing / serving layer	Elasticsearch or a columnar warehouse (e.g., Druid, BigQuery)	Dashboards need sub-second query response, which raw data-lake files cannot provide without an indexed serving layer in front

Diagram

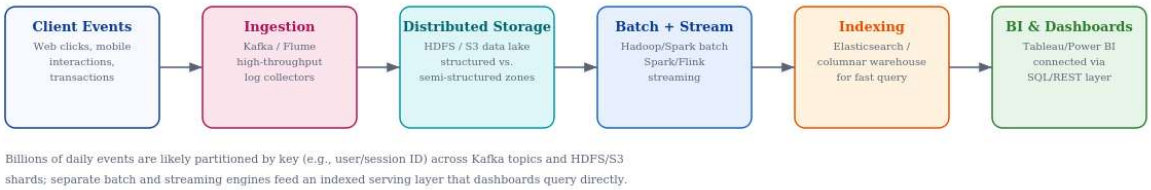

Fig 1: Probable architecture reverse-engineered from the described platform

Design Decisions Likely Made

- Partitioning by a high-cardinality key (user ID, session ID, or timestamp) across Kafka topics and storage shards, so writes and downstream processing scale horizontally rather than bottlenecking on one node
- A lambda-style split between batch and stream paths was probably chosen so slower, exhaustive batch reports and fast real-time dashboards can both be served without one workload slowing the other
- Dashboards and BI tools most likely connect to the indexed/warehouse layer via SQL or REST, never touching the raw data lake directly, keeping interactive query latency low

Q2. E-Commerce Personalization Pipeline (6 Marks)

Reverse-Engineered Understanding

Tracking cart additions, abandoned checkouts, product views, and payments across regions is a behavioural-event problem layered on top of a transactional one. The requirement for real-time personalisation is the strongest clue here: recommendations must react within a session, which is only possible if session/clickstream state is held somewhere far faster than a data warehouse — almost certainly an in-memory cache alongside the event stream.

Inferred Components and Technical Evidence

Inferred Component	Likely Technology	Evidence / Reasoning from Scenario
Event streaming	Apache Kafka, partitioned by region/user	"Across regions" and multiple event types (cart, view, checkout, payment) suggest a multi-topic Kafka setup, partitioned so regional load balances independently
Session/profiling store	Redis or similar in-memory key-value store	Real-time personalisation needs millisecond reads of current session state — a data warehouse query is far too slow for this
Distributed query engine	Spark SQL or Presto/Trino	Merging structured order records with semi-structured clickstream logs for analytics is a classic distributed-

		join workload too large for a single database
ML recommendation stage	Feature store + collaborative-filtering or embedding-based model	"Recommendation workflow" implies a model consuming engineered features (recent views, cart contents) rather than raw logs directly
Caching layer	CDN/application cache for product and recommendation results	Reduces repeated computation of the same recommendation for high-traffic products/pages
KPI dashboards	BI tool reading pre-aggregated marts	KPI monitoring almost always reads from summarised tables refreshed on a schedule, not raw event streams, to keep dashboards responsive

Diagram

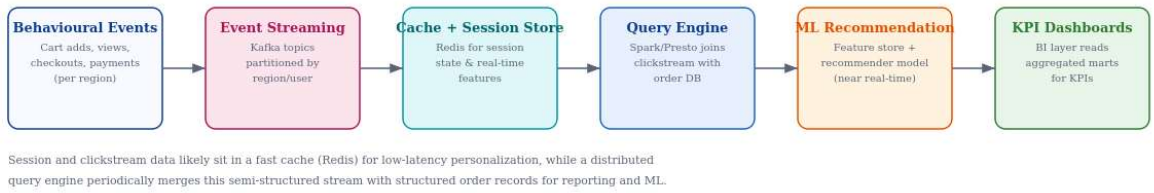

Fig 2: Probable architecture behind the e-commerce personalization pipeline

Design Decisions Likely Made

- Session data was probably kept in a fast cache separate from the durable event log, trading some durability for the low latency personalisation requires
- Fault tolerance likely comes from Kafka's replicated partitions and consumer-group rebalancing, so a failed consumer does not lose events, only delays their processing
- Structured order data and semi-structured browsing logs were likely merged downstream (batch/near-real-time join) rather than forced into one schema upfront, preserving the natural shape of each source

Q3. Healthcare Big Data System (Disease Prediction & Monitoring) (6 Marks)

Reverse-Engineered Understanding

Patient histories, diagnostic images, wearable streams, and prescriptions are four fundamentally different data shapes — tabular records, large binary images, continuous sensor readings, and structured pharmacy data — which is a strong signal that no single database technology was used. Combined with the need to connect hospitals, labs, and insurers, this points to a hybrid, interoperability-standard-driven architecture rather than a simple centralised database.

Inferred Components and Technical Evidence

Inferred Component	Likely Technology	Evidence / Reasoning from Scenario
Hybrid database architecture	Relational DB (structured records) + object storage (images) + NoSQL/document store (semi-structured notes)	Diagnostic images alone are too large and unstructured for a relational schema, while patient history is naturally tabular — a hybrid split is the practical fit
Data integration tools	HL7/FHIR interface engines and ETL connectors	"Connecting hospitals, laboratories, and insurance systems" is precisely

		what healthcare interoperability standards like HL7/FHIR exist to solve
Streaming framework	Kafka + Flink/Spark Streaming for wearable data	Continuous wearable sensor streams require the same low-latency stream processing pattern seen in other IoT-style scenarios
ML for disease prediction	Time-series/classification models trained on combined structured + streaming features	Predicting disease risk from ongoing monitoring implies a model consuming both historical records and live sensor trends
Privacy & compliance	Field-level encryption, role-based access control, audit logging (HIPAA/GDPR-style controls)	Healthcare data is among the most regulated data types, so encryption and access logging are near-certain requirements, not optional extras

Diagram

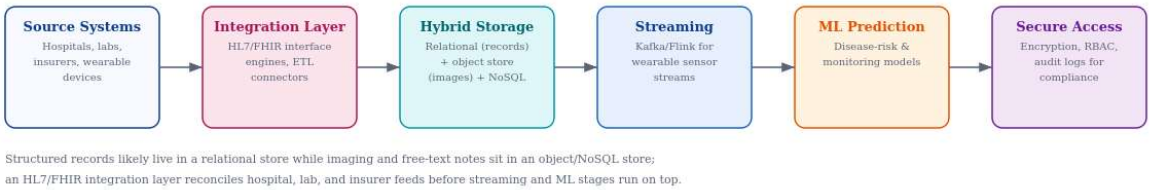

Fig 3: Probable hybrid architecture behind the healthcare big data system

Design Decisions Likely Made

- Structured and unstructured data were likely kept in separate, purpose-fit stores rather than one database, since forcing images into a relational schema would be both inefficient and impractical
- Encryption was probably applied both at rest (stored records/images) and in transit (between hospitals, labs, and the platform), since a single unencrypted hop would break compliance
- Access to individual patient records was likely scoped tightly by role (clinician vs. insurer vs. researcher), with population-health analytics run only on de-identified or aggregated data

Q4. Smart City Operational Intelligence Platform (6 Marks)

Reverse-Engineered Understanding

Continuous feeds from traffic sensors, CCTV metadata, weather, and public transport across a city are inherently distributed and location-bound, which strongly implies edge pre-processing near the sensors (to avoid streaming raw video/full sensor data citywide) and a centralised store built around time and location rather than generic keys. Congestion prediction for urban planning further implies both live streaming analytics and longer-term batch analysis are combined, not just one or the other.

Inferred Components and Technical Evidence

Inferred Component	Likely Technology	Evidence / Reasoning from Scenario
Edge computing	Edge gateways/nodes performing local filtering and aggregation	Raw CCTV and per-second sensor feeds citywide would overwhelm central bandwidth — pre-filtering at the edge (sending only metadata/events) is the practical choice

Distributed messaging	Kafka or MQTT (common for IoT-scale telemetry)	Continuous multi-source feeds (traffic, weather, transit) at city scale need a durable, publish-subscribe backbone rather than direct point-to-point connections
Centralized storage	Time-series database (e.g., InfluxDB-style) + geospatial database	Congestion prediction is inherently a "what, where, when" query — a combination of time-series and location-aware storage fits this far better than a generic relational store
Congestion analytics	Streaming engine (Flink/Spark Streaming) for live prediction + batch jobs for historical patterns	"Batch and real-time analytics combined" is stated directly in the scenario, matching a lambda-style split
Security / access control	Role-based access, anonymisation of CCTV metadata, network segmentation for critical infrastructure feeds	Citizen and infrastructure data sensitivity means raw feeds (especially CCTV) likely need anonymisation and restricted access by role

Diagram

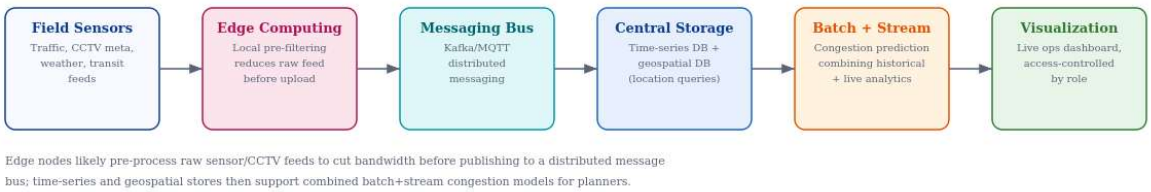

Fig 4: Probable architecture behind the smart city platform

Design Decisions Likely Made

- Edge pre-processing was likely chosen specifically to reduce the bandwidth and central storage cost of continuous CCTV and sensor feeds, sending events/metadata rather than raw streams
- Geospatial indexing was probably built in from the start, since congestion prediction and location-aware queries are core use cases, not an afterthought bolted onto a generic database
- Live operational dashboards likely read from the streaming layer directly for immediacy, while urban-planning reports likely read from the batch/historical layer for completeness

Q5. Healthcare Analytics System (Population-Level & Risk Scoring) (6 Marks)

Reverse-Engineered Understanding

This scenario is closely related to Q3 but emphasises population-level analysis and diagnosis risk scoring rather than real-time disease monitoring, which shifts the likely design toward distributed batch processing and interoperability-driven ETL rather than pure streaming. Integrating patient records, wearable readings, lab reports, and appointment histories across hospitals and clinics again implies a hybrid storage design, but here the analytics goal (population studies, risk scores) points to a strong distributed-processing layer sitting on top of that storage.

Inferred Components and Technical Evidence

Inferred Component	Likely Technology	Evidence / Reasoning from Scenario

Hybrid storage architecture	Relational store (records, appointments) + document/NoSQL store (lab reports, wearable readings)	Appointment histories and patient records are naturally tabular, while lab reports and wearable data vary in structure and volume, fitting a document-style store better
ETL & interoperability standards	HL7/FHIR-based ETL pipelines connecting hospitals and clinics	"Interoperability standards connecting hospitals and clinics" is a direct pointer to HL7/FHIR, the industry-standard healthcare data exchange format
Real-time monitoring	Lightweight streaming layer (Kafka + Spark Streaming) for wearable feeds	Even though the emphasis is population-level, "real-time monitoring" of wearables still implies a streaming component alongside the batch path
Distributed processing	Apache Spark for population-level aggregation and model training	Studying patterns across many patients at once is exactly the kind of large-scale aggregation Spark's distributed processing model is built for
Risk-scoring ML	Classification/regression models trained on combined structured + lab + wearable features	Diagnosis risk scoring and treatment recommendation is a supervised learning task that needs a rich, merged feature set from multiple sources
Governance	Encryption, consent tracking, and audit trails for every data access	Population health studies still touch individual patient data, so compliance controls from Q3 apply equally here

Diagram

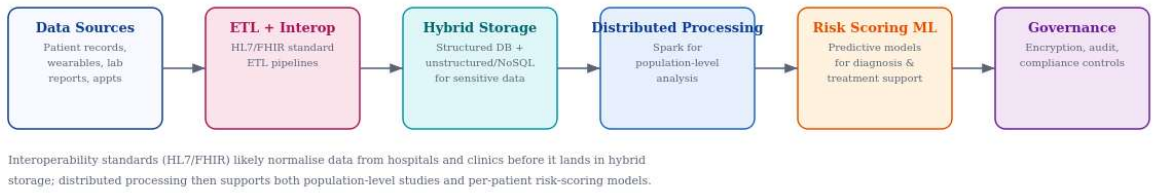

Fig 5: Probable architecture behind the population-level healthcare analytics system

Design Decisions Likely Made

- ETL pipelines were probably built around HL7/FHIR mappings specifically so new hospitals/clinics can be onboarded without custom point-to-point integration for each one
- Population-level analysis likely runs on de-identified or aggregated extracts processed by Spark, keeping raw identifiable records isolated from the broader analytics workload
- Risk-scoring models were probably retrained on a scheduled batch cycle (rather than continuously) since population-level patterns change more slowly than the real-time monitoring needs of Q3

Rubric Self-Check

Each answer above infers specific components directly from stated evidence in the scenario rather than guessing generically (Inference Accuracy), places those components into a coherent end-to-end big data architecture with a supporting diagram (Understanding of Architecture), ties every inferred technology back to a concrete phrase or requirement from the prompt (Use of Technical Evidence), goes beyond naming tools

to explain the design trade-offs likely behind each choice (Depth of Analysis), and follows a consistent structure of headings, evidence tables, and diagrams throughout (Clarity).